

Lecture 10:

Traditional ML vs Deep Learning

Vegard Høghaug Larsen

GRA4160: Predictive modelling with machine learning

March 11th 2026

Plan for today:

- ▶ Traditional ML vs Deep Learning - Overview
- ▶ Practical applications and model comparisons
- ▶ Tools used in practice
- ▶ Emerging trends in machine learning
- ▶ Feedback and questions on mini project

Traditional ML Models:

- ▶ Algorithms: k -NN, decision trees, logistic regression, SVMs
- ▶ Manual feature selection required
- ▶ Interpretable, good for structured data, smaller datasets
- ▶ Example: Decision trees as a flowchart

Deep Learning:

- ▶ Multi-layer neural networks
- ▶ Automatic feature learning from data
- ▶ Excellent with large datasets, images, text, audio
- ▶ High accuracy, high computational cost, low interpretability
- ▶ Often "black-box" models

- ▶ Traditional ML models plateau quickly — adding more data beyond a threshold yields diminishing returns
- ▶ Deep learning models scale better with data: performance continues to improve with larger datasets
- ▶ With **small data**, traditional ML often outperforms deep learning due to fewer parameters and less overfitting
- ▶ With **large data**, deep learning leverages its capacity to learn richer representations
- ▶ Key insight: the crossover point depends on task complexity and feature quality

“More data beats a cleverer algorithm, but only if the algorithm can exploit it.”

When to Choose Which?

- ▶ **Traditional ML:**

- ▶ Limited data
- ▶ Interpretability critical (e.g., credit decisions)

- ▶ **Deep Learning:**

- ▶ Large datasets available
- ▶ Complex, perceptual tasks (vision, NLP)
- ▶ Computational resources available

- ▶ Practice: Start simple (Logistic, Random Forest) then move to DL if needed

k-NN:

- ▶ Simple, memory-based, no explicit training
- ▶ Good for small-scale recommendations
- ▶ Slow at scale, limited to direct similarity

Neural Networks:

- ▶ Learns embeddings for user-item interactions
- ▶ Scalable, handles diverse data, highly accurate
- ▶ Used by Netflix, YouTube

Logistic Regression:

- ▶ Baseline, simple linear model, interpretable
- ▶ Effective with engineered features (e.g. TF-IDF)
- ▶ Limited nuance in complex texts

Deep Learning (CNNs, LSTMs, Transformers):

- ▶ Surpassed traditional methods on NLP tasks
- ▶ Learns word/context embeddings
- ▶ More data-intensive, computationally demanding

Decision Trees/Ensembles:

- ▶ Highly interpretable, suitable for small datasets
- ▶ Used for simple diagnostic rules
- ▶ Easy to validate by clinicians

Deep Learning:

- ▶ Human-level accuracy (X-rays, pathology slides)
- ▶ Harder to interpret, "black-box"
- ▶ Explainability (XAI) essential in practice

Random Forests:

- ▶ Requires extensive feature engineering
- ▶ Limited scalability and accuracy

CNNs:

- ▶ Revolutionized image recognition
- ▶ Learn hierarchical features directly from pixels
- ▶ State-of-the-art accuracy, computationally demanding

Ensemble Methods:

- ▶ Good with structured/tabular financial data
- ▶ Robust with smaller datasets
- ▶ Interpretable, less tuning required

Deep Learning:

- ▶ Powerful with large historical datasets
- ▶ Captures subtle, complex interactions
- ▶ Harder to interpret, extensive tuning required

Summary: Traditional ML vs Deep Learning

Aspect	Traditional ML	Deep Learning
Feature engineering	Manual	Automatic
Data requirement	Small–medium	Large
Interpretability	High	Low
Computational cost	Low	High
Structured data	Excellent	Good
Unstructured data	Limited	Excellent
Training time	Fast	Slow
Overfitting risk	Lower	Higher

Neither approach dominates — the best choice depends on the **data**, the **task**, and the **constraints**.

Scikit-learn:

- ▶ Traditional ML, quick prototyping, easy to use
- ▶ CPU-based, integrates with pandas, numpy

TensorFlow/PyTorch:

- ▶ Deep learning frameworks, GPU acceleration
- ▶ TensorFlow: Production deployments
- ▶ PyTorch: Research, dynamic graphs, HuggingFace integration

- ▶ **Hybrid Models:** Combining DL features with interpretable ML
- ▶ **Transfer/Self-supervised Learning:** Leveraging large, pre-trained models
- ▶ **Foundation Models:** GPT, Stable Diffusion, general-purpose models
- ▶ **AutoML/No-Code ML:** Automated model building, increased accessibility

- ▶ **Idea:** Take a model pre-trained on a large dataset and adapt it to a new, smaller task
- ▶ Common workflow:
 1. Start with a pre-trained model (e.g., ResNet for images, BERT for text)
 2. Freeze early layers that capture general features
 3. Fine-tune later layers on your specific dataset
- ▶ **Why it works:** Early layers learn generic features (edges, textures, word meanings) that transfer across tasks
- ▶ Dramatically reduces data requirements and training time
- ▶ Example: Fine-tuning a pre-trained BERT model for sentiment analysis can achieve strong results with just a few hundred labeled examples

When choosing between traditional ML and deep learning, consider:

1. **Data size:** <10k samples? Start with traditional ML
2. **Data type:** Tabular → tree-based methods; images/text/audio → deep learning
3. **Interpretability:** Required by regulation or stakeholders? Prefer traditional ML or add XAI methods
4. **Compute budget:** Limited GPU access? Traditional ML or smaller DL models
5. **Iteration speed:** Need quick experiments? Scikit-learn is faster to prototype
6. **Baseline first:** Always start with a simple model — you need a benchmark to justify complexity

Rule of thumb: if a gradient boosted tree (XGBoost/LightGBM) works well on your tabular data, deep learning will rarely beat it by much.